

# Linux Fundamental

## **Linux Homework Tasks**

### **Task 1: Soft Link & Hard Link**

- Learn the difference between soft links and hard links.
- Learn the commands to create both.
- Practice creating and deleting soft and hard links.
- Prepare for this as an interview question.

### **Task 2: adduser vs useradd**

- Learn the difference between adduser and useradd.
- Understand which command is preferred on Ubuntu/Linux and why.
- Create a test user using the recommended command.

### **Task 3: journalctl**

- Learn what journalctl is used for.
- Learn how to view system and service logs using journalctl.
- Practice checking logs for a specific service.

### **Task 4: Linux Command Cheat Sheet**

- Review the Linux command cheat sheet.
- Practice the important commands covered in the cheat sheet.
- Understand the purpose and basic usage of each command.

# Shell Scripting

# Shell Scripting Homework Task

## Task: System Information Script

Create a shell script that:

- Prints the current date.
- Prints the hostname.
- Prints the username.
- Prints the disk usage.
- Prints the running processes.
- Uses variables to store and use data.
- Takes user input using `read -p`.
- Creates a directory using `mkdir`.
- Creates a file using `touch`.
- Stores the running processes information in the file using `>` output redirection.

### Commands to Use

- `mkdir`
- `touch`
- `echo`
- `df`
- `ps`
- `read -p`
- Variables
- `>` output redirection

### Submission

- Create a public GitHub repository.
- Push the completed shell script to the repository.
- [Readme.md](#) file with all commands output.

# Networking Fundamentals

# Networking Homework Tasks

## Task 1

- Practice commands and repo shared in devops-hero github repo.

## Task 2

- Create an empty Markdown ( `.md` ) file.
- Execute the networking commands and add the output/screenshots to the file.
- Add a short explanation of what you understood about each command.

Git/GitHub

# Git Homework Tasks

## Task 1:

### `git commit -a -m`

- Practice `git commit -a -m "message"`.
- Understand the difference between `git commit -a -m` and `git commit -m`.
- Test both commands and observe the difference.

## Task 2: Git Cherry-Pick

- Create **2–4 commits** in the `main` branch.
- Use `git log` to view the commits.
- Create a new branch.
- Make **2–3 commits** in the new branch.
- Use `git log` to identify a specific commit.
- Cherry-pick one specific commit from the new branch into the `main` branch.
- Verify that the selected commit/change is now available in the `main` branch.

## Submission

- Take screenshots of your work **or** create an `.md` file showing the commands and output.
- Upload the screenshots or `.md` file to the GitHub repository.



# Docker Fundamental

# Docker Homework Tasks

## Task: Hello World Applications

Create simple **Hello World** web applications using Docker for:

- Node.js application (preferably React)
- Python application
- Java application
- Apache web server
- React application
- Nginx application

## Requirements

For each application:

- Create a separate folder:
  - `nodejs-app`
  - `python-app`
  - `java-app`
  - `Apache-app`
  - `React-app`
  - `nginx-app`
- Add the application code.
- Create a `Dockerfile`.
- Build the Docker image.
- Run the application using Docker.
- Verify that **Hello World** is displayed on a webpage.

## Submission

- Push all applications and Dockerfiles to your GitHub repository.

- Maintain the folder structure mentioned above.

# Dockerfiles & Images

# Docker Multi-Stage Build Homework

## Task 1: Run Multi-Stage Dockerfile

- Clone the repository containing the multi-stage Dockerfile.
- Build the Docker image using the multi-stage Dockerfile.
- Run a container from the image.
- Access the application running inside the container.
- Verify that the application displays:  
**Hello World from Docker multi-stage build**
- Verify the running container using `docker ps`.
- Confirm that the application is running on port `8080`.

## Task 2: Documentation

Create an `.md` file containing:

- Your name
- Your enrollment number
- Screenshot or output showing the application running successfully.
- Screenshot or output of `docker ps` showing the running container on port `8080`.

## Task 3: Docker Application Deployment

Deploy at least **3 different types of applications** using Docker, such as:

- Node.js
- Python
- Java

## Submission

- Upload the completed Markdown file to your GitHub repository.
- Include the required screenshots or command outputs as evidence.



# Docker Network

# Docker Networking & Volume Homework Tasks

## Task 1: Docker Container Networking

- Create 3 containers:
  - Frontend
  - Backend
  - Database
- Use Nginx or Alpine images for the frontend and backend.
- Use the MySQL image for the database.
- Create 3 different Docker networks.
- Add the backend container to 2 networks.
- Check connectivity between the containers.

## Task 2: Host Network

- Pull the Apache2 image from Docker Hub.
- Create an Apache2 container using the host network.
- Access the Apache website directly on port 80.

## Task 3: Bind Mount

- Create a folder on your local machine.
- Create an `index.html` file with **Hello students** as the content.
- Bind mount the folder to an Nginx container.
- Access the Nginx website and verify the content.
- Modify the `index.html` file.
- Verify that the changes are reflected without restarting the container.

## Task 4: Overlay Network

- Research Docker overlay networks.
- Understand their use cases.



- Understand how overlay networks work across multiple Docker hosts.

## Submission

- Take screenshots of the completed exercises.
- Add the screenshots to the [README.md](#) file in your GitHub repository.
- Complete any remaining exercises from the session.